

Cerberus - How Radix achieves infinite linear scalability while preserving atomic composability

Maintaining a Record of Transactions

14

To wrap up consensus and understand the data that nodes maintain, we revisit our example transaction.

Transaction (Tx)

- Shard 1: Shut down Alice's Token A substate
- Shard 2: Bring up Bob's Token A substate [Index: 0]
- Shard 3: Shut down Bob's Token B substate
- Shard 4: Bring up Carol's Token B substate [Index: 1]
- Shard 5: Shut down Carol's Token C substate
- Shard 6: Bring up Alice's Token C substate [Index: 2]
- Signed: Alice, Bob, Carol

Tx ID: ...979

We can see that the transaction contains all the details needed to create each substate in their respective shards...

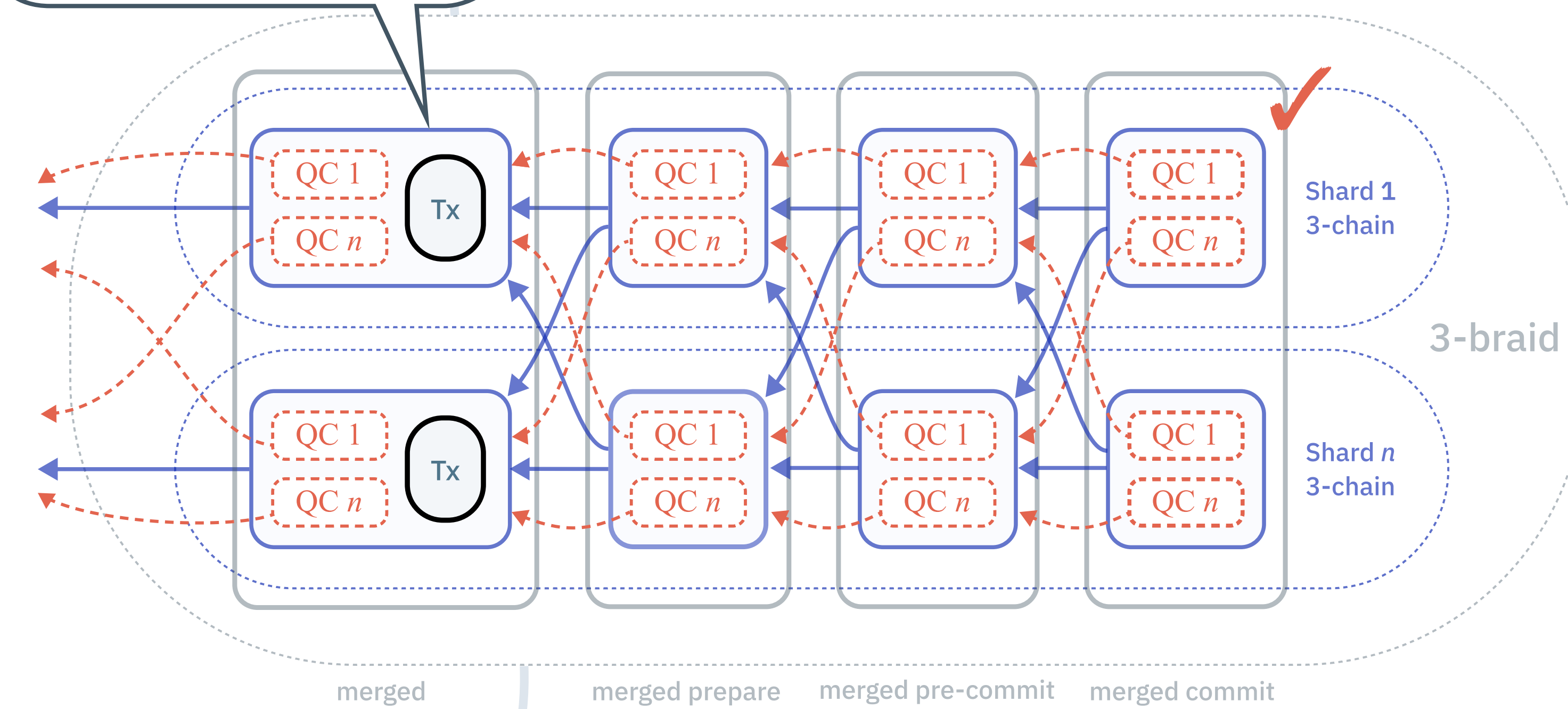
...and in order to write the transaction to the ledger, nodes need some way of coming to consensus between validator sets, across shards.

This is achieved through the unique "braiding" design of emergent Cerberus, where QCs and vertices for each shard are shared between all shards, and chained together with hashes.

Because of this, every node in every validator set has proof that $2f+1$ of vote weight in every shard approved each phase of consensus.

This design uniquely allows validator sets in Cerberus to come to consensus across shards - either all together, or not at all.

The merged proposal vertex includes the transaction and its substates.



But so far we've only covered how nodes verify and vote on transactions. We haven't covered one of the most integral parts of their responsibilities - maintaining a record of transactions.

So what data do nodes have to keep, in order to fulfill this duty?

Nodes store a copy of the whole transaction, which includes all the substates across all relevant shards.

They also store proof that $2f+1$ of vote weight approved the transaction across every shard, for every phase of consensus through the QCs in each vertex.

Each vertex is chained to the previous vertex via hashes - cryptographically tying every phase of consensus together.

Nodes store the transaction and proof that $2f+1$ of vote weight approved it

The merged proposal vertex includes the transaction and its substates.

The proposal vertices in the future transaction will point back to the commit QC vertex from our transaction.

Then, if a future transaction ever wants to use any one of the bring up substates created in our transaction, it grabs only the substate it needs, shuts it down by creating a new shut down substate, and creates new bring up substate(s) in new shards.

Relevant validator sets then come to consensus on new proposal vertices that point back to the commit QC vertex from our transaction. The validator set serving Shard 4 depicted here has all the data it needs to verify the future transaction as it has a complete copy of our transaction, and a complete copy of the future transaction.

And the cycle continues...

So that's a wrap on consensus. Now just one last thing - Sybil Resistance Through Proof of Stake.

The version of Cerberus described in this infographic series is scheduled to launch as part of the fully sharded Radix Xi'an release. Please visit www.radixdlt.com for details on the Radix roadmap.